

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №5

«Наследование. Виртуальные функции. Полиморфизм.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Постановка задачи:

1. Определить абстрактный класс
2. Определить иерархию классов, в основе которой будет находиться абстрактный класс (иерархия классов берется из лабораторной работы №4)
3. Определить класс Вектор, элементами которого будут являться указатели на объекты иерархии классов
4. Перегрузить для класса Вектор операцию вывода объектов с помощью потоков
5. В основной функции продемонстрировать перегруженные операции и полиморфизм класса Вектор.

Базовый класс:

Базовый класс:

ЧЕЛОВЕК (PERSON)

Имя (name) – string

Возраст (age) – int

Определить методы изменения полей.

Создать производный класс STUDENT, имеющий поля Предмет – string

Оценка – int. Определить методы изменения полей и метод, выдающий сообщение о неудовлетворительной оценке.

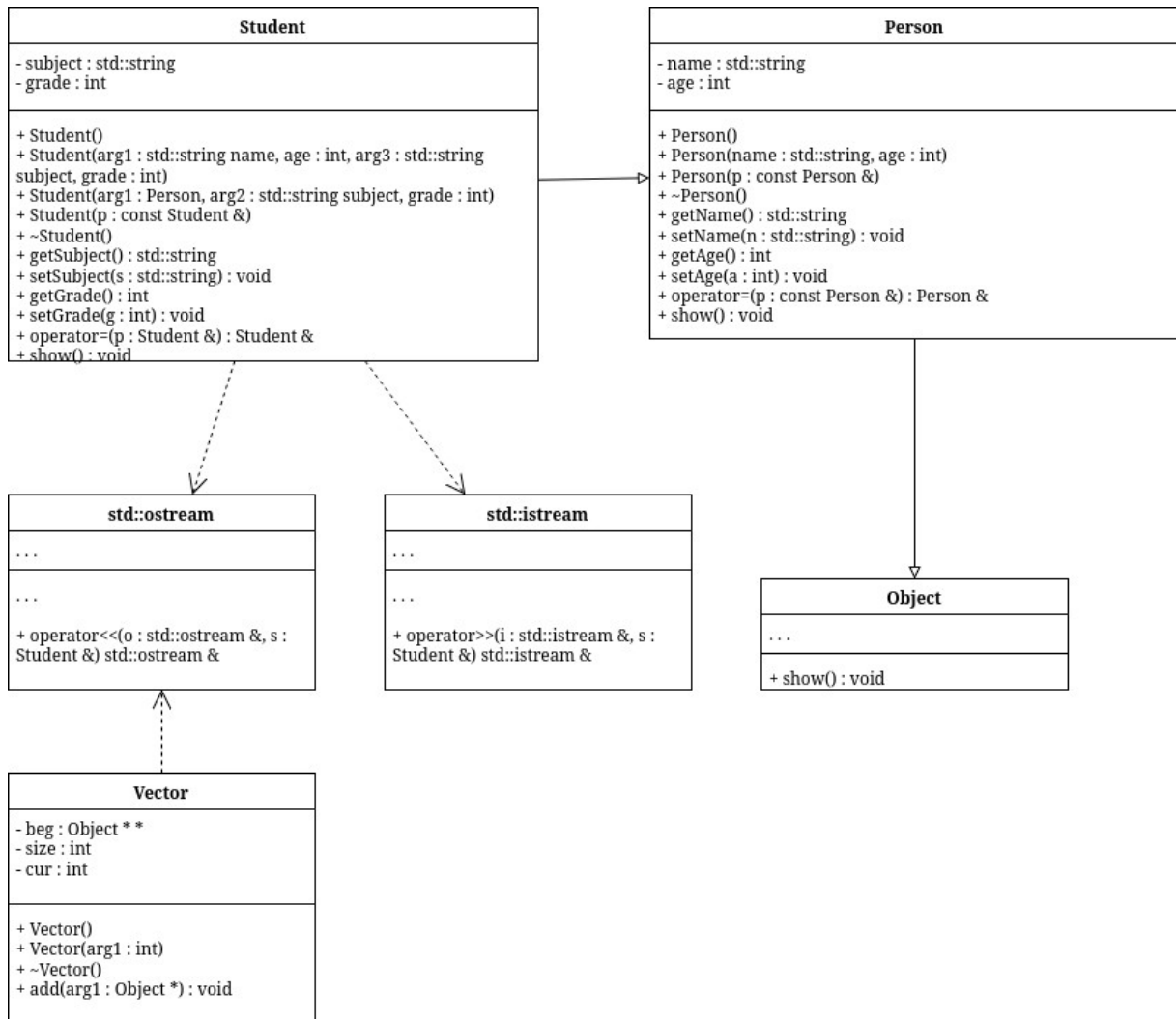
АНАЛИЗ

1. Создадим абстрактный класс Object, имеющий чистую виртуальную функцию show().
2. Создадим класс Person : Object()

3. Создадим класс Teacher : Person

4. Класс vector будет оперировать указателями на Object и методом show().

UML ДИАГРАММА



КОД Object.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_OBJECT_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_OBJECT_H

class Object {
public:
    virtual void show() = 0;
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_OBJECT_H
```

КОД Person.h

```
//
// Created by localuser on 4/8/26.
//

#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
#include <string>
#include "Object.h"

class Person : public Object {
    std::string name;
    int age;

public:
    Person();
    Person(std::string, int);
    Person(Person &);
    ~Person();
    const std::string getName();
    void setName(std::string);
    const int getAge();
    void setAge(int);
    Person &operator=(Person const &p);
    friend std::ostream &operator<<(std::ostream &, const Person &);
    friend std::istream &operator>>(std::istream &, Person &);
    void show() override;
};
```

```
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
```

КОД Person.cpp

```
//  
// Created by localuser on 4/8/26.  
//  
  
#include "Person.h"  
  
#include <iostream>  
  
Person::Person() {  
    name = "Unknown";  
    age = 18;  
}  
  
Person::Person(std::string s, int a) {  
    name = s;  
    age = a;  
}  
  
Person::Person(Person &p) {  
    name = p.name;  
    age = p.age;  
}  
  
Person::~~Person() {  
    name = "Unknown";  
    age = 18;  
}  
  
const std::string Person::getName() {  
    return name;  
}  
  
void Person::setName(std::string s) {  
    name = s;  
}  
  
const int Person::getAge() {  
    return age;  
}
```

```

void Person::setAge(int a) {
    age = a;
}

Person &Person::operator=(Person const &p) {
    name = p.name;
    age = p.age;
    return *this;
}

void Person::show() {
    std::cout << *this;
}

std::ostream &operator<<(std::ostream &stream, const Person &p) {
    stream << p.name << " : " << p.age << '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, Person &p) {
    std::cout << "Name? ";
    stream >> p.name;
    std::cout << "Age? ";
    stream >> p.age;
    return stream;
}

```

KOД Student.h

```

//
// Created by localuser on 4/8/26.
//

#ifdef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#include <string>
#include "Person.h"

class Student : public Person {
    std::string subject;
    int grade;
public:
    Student();
    Student(std::string name, int age, std::string subject, int grade);

```

```

Student(Person, std::string subject, int grade);
Student(Student&);
~Student();
std::string getSubject();
void setSubject(std::string);
int getGrade();
void setGrade(int);
Student &operator=(Student &p);
friend std::ostream &operator<<(std::ostream &, Student &);
friend std::istream &operator>>(std::istream &, Student &);
void show();
bool isOk() {return grade > 2;};
};

```

```

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H

```

КОД Student.cpp

```

//
// Created by localuser on 4/8/26.
//

```

```

#include "Student.h"

```

```

#include <iostream>

```

```

Student::Student() {
    subject = "Unknown";
    grade = 5;
}

```

```

Student::Student(std::string name, int age, std::string subject, int grade) : Person(name, age) {
    this->subject = subject;
    this->grade = grade;
}

```

```

Student::Student(Person p, std::string subject, int grade) : Person(p){
    this->subject = subject;
    this->grade = grade;
}

```

```

Student::Student(Student & p) {
    subject = p.subject;
}

```

```

    grade = p.grade;
}

Student::~~Student() {
    subject = "Unknown";
    grade = 5;
}

std::string Student::getSubject() {
    return subject;
}

void Student::setSubject(std::string s) {
    subject = s;
}

int Student::getGrade() {
    return grade;
}

void Student::setGrade(int a) {
    grade = a;
}

Student &Student::operator=(Student &p) {
    this->setName(p.getName());
    this->setAge(p.getAge());
    subject = p.subject;
    grade = p.grade;
    return *this;
}

std::ostream &operator<<(std::ostream &stream, Student &p) {
    stream << p.getName() << " : " << p.getAge() << " / " << p.getSubject() << " : " << p.getGrade();
    if (!p.isOk()) stream << " not ok :(";
    stream << '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, Student &p) {
    std::cout << "name? ";
    std::string s;
    stream >> s;
    p.setName(s);
    std::cout << "age? ";

```



```

int a;
stream >> a;
p.setAge(a);
std::cout << "subject? ";
stream >> p.subject;
std::cout << "grade? ";
stream >> p.grade;
return stream;
}

```

```

void Student::show() {
    std::cout << *this;
}

```

КОД Vector.h

```

//
// Created by localuser on 4/8/26.
//

#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_VECTOR_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_VECTOR_H
#include "Person.h"

class Vector {
    Object **beg;
    int size;
    int cur;
public:
    Vector();
    Vector(int);
    ~Vector();
    void Add(Object *);
    friend std::ostream &operator<<(std::ostream &out, const Vector &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_VECTOR_H

```

КОД Vector.cpp

```

//
// Created by localuser on 4/8/26.
//

```

```

#include "Vector.h"

#include <iostream>

Vector::Vector() {
    beg = nullptr;
    size = 0;
    cur = 0;
}

Vector::~Vector() {
    if (beg != nullptr) delete [] beg;
    beg = 0;
}

Vector::Vector(int n) {
    beg = new Object *[n];
    cur = 0;
    size = n;
}

void Vector::Add(Object *p) {
    if (cur < size) {
        beg[cur] = p;
        cur++;
    }
}

std::ostream &operator<<(std::ostream &out, const Vector &v) {
    if (v.size == 0)
        std::cout << "Empty\n";
    Object **p = v.beg;
    for (int i = 0; i < v.cur; i++)
        p[i]->show();

    return out;
}

```

КОД main.cpp

```

#include <iostream>
#include "Student.h"
#include "Vector.h"

```

```

using namespace std;

void f1(Person &p) {
    p.setName("John");
}

void test(Object *o) {
    o->show();
}

int main() {
    Person person("Alex", 20);
    Student student(person, "Math", 5);
    f1(student);
    person.show();
    student.show();
    student.setGrade(2);
    Vector v(100);
    v.Add(&student);
    v.Add(&person);
    cout << v;
}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Alex : 20

John : 20 / Math : 5

John : 20 / Math : 2 not ok :(

Alex : 20

ВОПРОСЫ

1. Какой метод называется чисто виртуальным? Чем он отличается от виртуального метода?

Метод, при объявлении не имеющий и не имеющий возможность иметь реализацию. Используется ключевое слово = 0

2. Какой класс называется абстрактным?

Класс, имеющий не переопределенные (или заданные) хотя-бы одну чистую виртуальную функцию.

3. Для чего предназначены абстрактные классы?

Для описания унифицированных интерфейсов работы для различных классов.

4. Что такое полиморфные функции?

Это возможность вызывать один и тот же интерфейс для объектов разных типов.

5. Чем полиморфизм отличается от принципа подстановки?

Это механизм языка, позволяющий работать с объектами разных типов через общий интерфейс.

6. Привести примеры иерархий с использованием абстрактных классов.

```
class Shape {  
public:  
    virtual double area() = 0;  
};  
  
class Circle : public Shape {};  
class Rectangle : public Shape {};  
class Triangle : public Shape {};
```

7. Привести примеры полиморфных функций.

```
void getArea(Shape* s) {  
    cout << s.area();  
}
```

8. В каких случаях используется механизм позднего связывания?

При работе с виртуальными функциями, когда тип объекта известен только во время выполнения;